

The code for **igos-build** is a clean, well-structured Python build orchestrator tailored for an LFS/BLFS-style from-source Linux distribution. It strikes a good balance between automation, traceability, and safety for an offline chroot environment. Overall, it's production-ready for its intended scope (hundreds of packages, reproducible builds, resume capability), with thoughtful design decisions that align with LFS conventions (e.g., heavy use of DESTDIR, multi-pass toolchain handling) and Slackware-inspired tracking.

I'll address your specific review points first, then provide broader feedback on architecture, quality, and suggestions.

## 1. Dependency Resolution and Cycle Detection

The implementation in `graph.py` is solid. It uses a standard **Kahn's algorithm** (BFS with in-degrees) for topological sort, which is efficient ( $O(V + E)$ ) and naturally detects cycles when not all nodes are processed.

- **Strengths:**
  - Merges build, host, and runtime dependencies into a single "must build before" relation — correct for a from-source distro where everything is built in order.
  - Stable sorting within "waves" using tier priority + pass\_number + name ensures deterministic, human-friendly order (toolchain first, then core/base/desktop). This is better than a naive sort.
  - `_find_cycle` does a reasonable DFS to report one cycle (good enough for debugging; most cycles are small).
  - `strict=False` mode in `build_graph` is pragmatic for incremental template development.
- **Potential improvements:**
  - The cycle finder only searches remaining nodes and may not always return the *minimal* cycle, but that's rarely an issue.
  - No explicit support for versioned or alternative dependencies (fine for LFS, where deps are usually exact package names).
  - Consider adding a `graph.dot()` export for visualization (e.g., via `graphviz`) during development — very helpful for large graphs (~470 packages).

**Verdict:** Topological sort is correct; cycles are properly reported. No major issues.

## 2. DESTDIR Staging Safety

The pre-deploy check in `pkg_deploy()` is a smart, targeted safeguard. It explicitly detects when a package would stage a real directory over root symlinks (`/lib` → `usr/lib`, `/bin` → `usr/bin`, etc.) — a classic way to break the dynamic linker or filesystem layout in a merged-`/usr` system.

- The tar pipe (`tar -C staging -cf - . | tar -C / -xf - --no-overwrite-dir --keep-directory-symlink`) is efficient and reasonably safe.
- Creating mirror symlinks and `usr/*` dirs in staging (for non-`direct_install` packages) helps many make `install / ninja install` targets behave correctly.
- For `direct_install` packages (multi-pass like GCC/glibc), you bypass DESTDIR entirely and use filesystem diff — pragmatic but riskier.

### Concerns:

- The check only looks for a small hardcoded list (`lib`, `lib64`, `bin`, `sbin`). What about `/etc`, `/var`, or other top-level items that could cause issues?
- Tar extraction with `--keep-directory-symlink` helps, but malicious or buggy packages could still do damage if they install unexpected symlinks or devices.
- No UID/GID remapping or ownership checks (LFS usually runs as root in chroot, so this is expected, but document it).

**Is it sufficient?** For well-behaved LFS/BLFS packages, yes — especially since you control the templates. It's better than many ad-hoc build systems. For extra safety, you could add a configurable "dangerous\_paths" list or a full dry-run mode that simulates the tar extract without writing.

## 3. Error Handling

Error handling is generally excellent and consistent:

- Almost every external command (subprocess.Popen in `run_command`, `tar`, `sha256sum`, etc.) checks return codes.
- Failures are logged verbosely with context (package, phase, command, log file location).
- `halt_on_failure=True` (default) stops cleanly.
- Validation checks respect the fatal flag.
- `BuildLogger` and `SummaryLogger` provide clear per-package and overall summaries.

### Minor gaps:

- Some subprocess.run(..., capture\_output=True) calls log errors but could miss edge cases (e.g., very large stderr).
- In extract\_source, zip extraction and bundled dep handling have try/except, but the generic Exception is broad — consider catching more specific ones.
- No retry logic for transient issues (not needed in offline chroot).
- Silent failures are rare; the main risk is an unhandled exception bubbling up from YAML parsing or graph building (already caught in main()).

Overall, errors are not silently swallowed in normal paths. Good job.

## 4. --skip-built Logic

Checking only for manifest existence (/var/lib/igos/packages/<name>-<version>) is a **reasonable trade-off** for speed and simplicity, especially since you already run pkg\_verify at install time.

- **Pros:** Fast resume after interruptions; avoids re-verifying everything (which, as you noted, can false-positive due to post\_install side effects like moving files).
- **Cons:** Doesn't detect corrupted installs, partial deploys, or changes made outside the build system. No content checksums on the manifest itself.

**Recommendation:** Keep the current behavior as default for --skip-built. Optionally add a --verify or --force-rebuild flag that runs full pkg\_verify + optional content hashes. For critical packages (toolchain), you could force verification.

This matches the spirit of Slackware's simple tracking while adding your own safety net.

## 5. Security

Given the offline chroot + pre-downloaded sources, the attack surface is small, but several areas deserve attention:

- **Command injection:**
  - run\_command uses shell=True with fully controlled cmd strings you generate (from templates or build styles). Package names/versions flow into env vars (PKG\_VERSION, etc.) and paths, but not directly into shell.

- However, build.sh scripts (sourced and calling user-defined functions) and configure\_flags / patches lists are trusted input. If a malicious template sneaks in, it's game over anyway.
- wget call in extract\_source (if triggered) uses quoted URLs — safe.
- **Path traversal:**
  - Tar extraction uses --strip-components=1 — good, but classic tarfiles can still have ../ entries (Python's tarfile has known historical issues, though you're using tar binary for most cases).
  - Staging dir construction (pkg\_staging / f"{pkg.name}-{pkg.version}") and manifest/archive paths use controlled names — no obvious traversal.
  - fs\_snapshot and diff-based tracking walk real filesystem paths — safe in chroot.
- **Other:**
  - YAML parsing with yaml.safe\_load — correct (avoids arbitrary code).
  - No network access during normal builds (except optional download fallback).
  - Root execution in chroot is inherent risk; the DESTDIR safety check mitigates one common foot-gun.

**Verdict:** Low risk for intended use. To harden further:

- Switch tar extraction to Python's tarfile with explicit filter='data' (Python 3.12+) or add --anchored --no-wildcards flags.
- Sanitize package names more aggressively (alphanumeric + hyphen/underscore).
- Consider running non-toolchain builds as non-root if feasible (LFS/BLFS recommends this where possible).

## 6. General Code Quality, Maintainability, Edge Cases

### Strengths:

- Excellent modularity: parser → graph → styles → builder → logger. Clear separation of concerns.
- Dataclasses + type hints make the code readable.
- Logging is comprehensive (per-package timestamped logs + streaming output + summary).
- Handles LFS-specific needs well: multi-pass (pass\_number, direct\_install, target\_triple), bundled deps, custom build.sh.

- Dry-run and verbose modes are very useful for auditing.
- Template validation is strict and helpful.

### Areas for improvement:

- **Argument parsing:** Manual `sys.argv` checks are brittle (order-dependent, no help text, duplicates possible). Switch to `argparse` (or `click/typer`) for proper flags, defaults, and `--help`.
- **Error handling in `main()`:** Good top-level catches, but more granular exceptions (e.g., for `sources_dir` issues) would help.
- **Platform assumptions:** `platform.machine() == "x86_64"` for `lib64` — consider making this configurable or detecting via `uname`.
- **Edge cases:**
  - Packages with no source (`source: []`).
  - Zero or multiple sources (you handle primary + bundled, but document expectations).
  - Very large packages or logs (streaming helps, but consider log rotation or size limits).
  - Empty manifests or zero-file packages.
  - YAML with non-string keys/values (your parser mostly coerces to `str/int`).
- **Performance:** For 470 packages, graph building and sorting are negligible. Extraction/checksum is the bottleneck (fine).
- **Testing:** No tests visible in the provided code. Unit tests for parser/graph/styles would be high value. Integration tests in a minimal chroot would be ideal.
- **Documentation:** Inline comments are good; consider adding a README with architecture overview, how to add a new style, and template examples.

### Minor cleanups:

- Some duplication in manifest generation (staged vs. diff) — could share more helper logic.
- `build_env` is long; consider splitting env setup for tracked vs. non-tracked.
- In custom style, `post_install` is nicely separated (runs on live FS after deploy).

## Overall Thoughts

This is one of the cleaner custom LFS build systems I've seen. It avoids the fragility of pure shell scripts while keeping the build steps transparent and auditable (shell commands are

explicit in styles or build.sh). The combination of YAML templates + optional build.sh is flexible without being overly complex.

It feels inspired by tools like Void Linux's xbps-src or Gentoo's Portage but simplified for a pure from-source, offline workflow. The tracking system (manifests + archives + verify) adds real package management value on top of raw LFS.

**Priority suggestions** (in rough order):

1. Switch to argparse in `__main__.py`.
2. Harden tar extraction (explicit filters or flags).
3. Add a few more safety checks in `pkg_deploy` (e.g., for other top-level paths).
4. Document the build directory layout, tracked vs. non-tracked modes, and how `direct_install` is intended.
5. Consider a `--verify-all` mode for post-build audits.

Minor issues aside, this looks maintainable and should scale well as you add the remaining ~470 templates. Great work — it's clear a lot of thought went into reliability and resume-ability.

If you'd like me to dive deeper into any file, suggest refactors, or review specific build.sh examples (not provided), just share more details!